

LAB #5 Directory Access, Hard Links, and Symbolic Links

1. *Apply directory access system calls to display and interpret directory contents.*
2. *Analyze file attributes using stat() and lstat() system calls.*
3. *Differentiate between hard links and symbolic links using inode analysis.*
4. *Demonstrate the ability to create, verify, and manage file links in a Linux environment.*

Pre-Lab Theory:

SYNOPSIS

```
#include <dirent.h>
#include <dirent.h>
#include <sys/stat.h>
char *getcwd(char *buf, size_t size);
DIR *opendir(const char *dirname);
struct dirent *readdir(DIR *dirp);
int closedir(DIR *dirp);
void rewinddir(DIR *dirp);
int lstat(const char *restrict path, struct stat *restrict buf);
int stat(const char *restrict path, struct stat *restrict buf);
```

The struct stat structure, which is defined in sys/stat.h, contains at least the following members.

```
dev_t st_dev; /* device ID of device containing file */
ino_t st_ino; /* file serial number */
mode_t st_mode; /* file mode */
nlink_t st_nlink; /* number of hard links */
uid_t st_uid; /* user ID of file */
gid_t st_gid; /* group ID of file */
off_t st_size; /* file size in bytes (regular files) */
/* path size (symbolic links) */
time_t st_atime; /* time of last access */
time_t st_mtime; /* time of last data modification */
time_t st_ctime; /* time of last file status change */
```

LAB TASKS

Task 1: Apply the `getcwd()` system call to obtain and display the current working directory path.

Write the full directory path displayed by your program.

Task 2: Implement and demonstrate directory traversal using `opendir()`, `readdir()`, and `closedir()`.

Output:

Task 3: Differentiate between files and directories using file type macros with stat().

Code Outline:

```
#include <stdio.h>
#include <dirent.h>
#include <sys/stat.h>

int main() {
    DIR *dir;
    struct dirent *entry;
    struct stat fileStat;
    dir = opendir(".");
    while ((entry = readdir(dir)) != NULL) {
        stat(entry->d_name, &fileStat);
        if (S_ISDIR(fileStat.st_mode))
            printf("%s --> Directory\n", entry->d_name);
        else
            printf("%s --> File\n", entry->d_name);
    }
    closedir(dir);
    return 0;
}
```

- Paste the program output showing which items are identified as files or directories.

- Write the names of at least two directories detected by your code.

Task 4: Apply the `stat()` system call to extract and analyze metadata such as permissions, owner, and link count.

- Paste the output showing metadata for test.txt.

Task 5: Demonstrate the creation of hard and symbolic links and evaluate their structural differences using inode numbers.

Code Outline:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/stat.h>

int main() {
    link("test.txt", "hardlink_test.txt");
    symlink("test.txt", "softlink_test.txt");
    printf("Hard and soft links created successfully.\n");
    struct stat hardStat, softStat;
    stat("hardlink_test.txt", &hardStat);
    lstat("softlink_test.txt", &softStat);
    printf("Hard Link inode: %ld\n", hardStat.st_ino);
    printf("Soft Link inode: %ld\n", softStat.st_ino);
    return 0;
}
```

- Paste the output showing successful creation of both links.
- Record the inode numbers displayed for original file, hard link, and soft link.
- Write which link remains accessible after deleting the original file.

Task 6: *Extend the Task 2 to implement and demonstrate recursive directory traversal using `opendir()`, `readdir()`, and `closedir()` to display the contents of the current directory and all its subdirectories up to the last level in the hierarchy.*

*Paste the output **showing full directory structure** printed by your program (including subdirectories).*

*Write the **name of the deepest subdirectory** found during execution.*